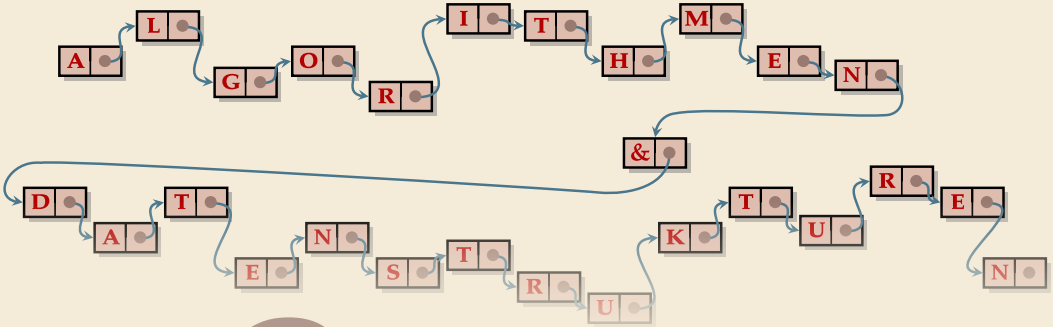


3 Mathematische Grundlagen

Algorithmen & Datenstrukturen · Sommersemester 2026

Prof. Dr. Sebastian Wild

CS210 (Sommersemester 2026)
Philipps-Universität Marburg
version `2026-04-27 10:09`

Outline

3 Mathematische Grundlagen

3.1 Beispiel: 3SUM
3.2 Folgen & Grenzwerte
3.3 Asymptotische Notation
3.4 Rechenregeln für Asymptotiken
3.5 Rekursiv definierte Folgen
3.6 Das Master-Theorem
3.7 Grundlagen Stochastik

Recap: Skalierbarkeit

Analogie: Architektur

- 1m Brücke: Holzplanke genügt!
- 10x $\rightsquigarrow$ Konstruktion nötig
- 1000x $\rightsquigarrow$ Meisterleistung der Bauingenieurskunst

Eine 1km Brücke ist nicht einfach ein 1000-fach größeres Brett.

Recap: Skalierbarkeit

Analogie: Architektur

- 1m Brücke: Holzplanke genügt!
- 10x $\rightsquigarrow$ Konstruktion nötig
- 1000x $\rightsquigarrow$ Meisterleistung der Bauingenieurskunst

Eine 1km Brücke ist nicht einfach ein 1000-fach größeres Brett.

→

→

→

Recap: Skalierbarkeit

Analogie: Architektur

- 1m Brücke: Holzplanke genügt!
- 10x $\rightsquigarrow$ Konstruktion nötig
- 1000x $\rightsquigarrow$ Meisterleistung der Bauingenieurskunst

Eine 1km Brücke ist nicht einfach ein 1000-fach größeres Brett.

2m

$\xrightarrow{\sim 10x}$

12m

$\xrightarrow{\sim 10x}$

170m

$\xrightarrow{\sim 10x}$

1280m

Recap: Skalierbarkeit

Analogie: Architektur

- 1m Brücke: Holzplanke genügt!
- 10x $\rightsquigarrow$ Konstruktion nötig
- 1000x $\rightsquigarrow$ Meisterleistung der Bauingenieurskunst

Eine 1km Brücke ist nicht einfach ein 1000-fach größeres Brett.

2m	$\xrightarrow{\sim 10x}$	12m	$\xrightarrow{\sim 10x}$	170m	$\xrightarrow{\sim 10x}$	1280m
€ 50 000	$\xrightarrow{\sim 10x}$	€ 350 000	$\xrightarrow{\sim 50x}$	€ 22 000 000	$\xrightarrow{\sim 200x}$	€ 4 500 000 000

(Kosten sind Schätzungen der heutigen Kosten)

Recap: Skalierbarkeit

Analogie: Architektur

- 1m Brücke: Holzplanke genügt!
- 10x $\rightsquigarrow$ Konstruktion nötig
- 1000x $\rightsquigarrow$ Meisterleistung der Bauingenieurskunst

Informatik

- Eingabegrößen in Informatiksystemen erreichen oft Faktor 1 000 000 000!

  1 Byte $\longrightarrow$ 1 GB

$\rightsquigarrow$ *Können uns keine Holzbretter leisten.*

Eine 1km Brücke ist nicht einfach ein 1000-fach größeres Brett.

$\longrightarrow$

$\longrightarrow$

$\longrightarrow$

2m	$\xrightarrow{\sim 10x}$	12m	$\xrightarrow{\sim 10x}$	170m	$\xrightarrow{\sim 10x}$	1280m
€ 50 000	$\xrightarrow{\sim 10x}$	€ 350 000	$\xrightarrow{\sim 50x}$	€ 22 000 000	$\xrightarrow{\sim 200x}$	€ 4 500 000 000

(Kosten sind Schätzungen der heutigen Kosten)

Wie quantifizieren wir “Skalierbarkeit”?

- ⇝ *Für Algorithmen (in erster Näherung)* ***Wachstum*** *der Kosten in Eingabegröße entscheidend.*
- Tricks in der Implementierung und bessere Hardware retten uns nicht
- tragen aber natürlich zu den Gesamtkosten bei
- ⇝ für Performance-Bottlenecks durch Experten noch etwas rauszuholen
- ⇝ den deutlich größeren Impact hat aber die Wahl der richtigen Algorithmen und Datenstrukturen

Wie quantifizieren wir "Skalierbarkeit"?

⇝ *Für Algorithmen (in erster Näherung)* ***Wachstum*** *der Kosten in Eingabegröße entscheidend.*

- Tricks in der Implementierung und bessere Hardware retten uns nicht
- tragen aber natürlich zu den Gesamtkosten bei

⇝ für Performance-Bottlenecks durch Experten noch etwas rauszuholen

⇝ den deutlich größeren Impact hat aber die Wahl der richtigen Algorithmen und Datenstrukturen

Wie können wir also Algorithmen bezüglich des ***Wachstums*** *der Kosten* ***vergleichen****?*

- Sprache für Performancecharakteristika: **Asymptotische Approximationen**
  O-Notation (plus Varianten und Verfeinerungen)

⇝ Damit müssen Sie souverän umgehen lernen.

Wie quantifizieren wir "Skalierbarkeit"?

⇝ *Für Algorithmen (in erster Näherung)* ***Wachstum*** *der Kosten in Eingabegröße entscheidend.*

- Tricks in der Implementierung und bessere Hardware retten uns nicht
- tragen aber natürlich zu den Gesamtkosten bei

⇝ für Performance-Bottlenecks durch Experten noch etwas rauszuholen

⇝ den deutlich größeren Impact hat aber die Wahl der richtigen Algorithmen und Datenstrukturen

Wie können wir also Algorithmen bezüglich des ***Wachstums*** *der Kosten* ***vergleichen****?*

- Sprache für Performancecharakteristika: **Asymptotische Approximationen**
  O-Notation (plus Varianten und Verfeinerungen)

⇝ Damit müssen Sie souverän umgehen lernen.

Dazu legen wir hier die Grundlagen und geben die formalen Definitionen.

3.1 Beispiel: 3SUM

3SUM

3SUM-Problem

- **Gegeben:** $A[0..n)$ with $A[i] \in \mathbb{Z}$ for $0 \leq i < n$.
- **Ziel:** Anzahl c von Tripeln aus A mit Summe 0

$$c \quad = \quad \left|\left\{(i,j,k) \;:\; 0 \leq i < j < k < n \;\wedge\; A[i] + A[j] + A[k] = 0\right\}\right|$$

Parameter:

- Eingabegröße: # Zahlen n

3SUM

3SUM-Problem

- **Gegeben:** $A[0..n)$ with $A[i] \in \mathbb{Z}$ for $0 \leq i < n$.
- **Ziel:** Anzahl c von Tripeln aus A mit Summe 0

$$c \;=\; \left|\left\{(i,j,k) \;:\; 0 \leq i < j < k < n \;\wedge\; A[i] + A[j] + A[k] = 0\right\}\right|$$

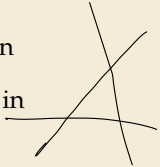
Parameter:

- Eingabegröße: #Zahlen n

Anwendungen:

3 Punkte auf einer Geraden

- kollineare Tripel von Punkten finden
- Tripel von Geraden finden, die sich in einem Punkt schneiden



- Dreiecke aus Punkten formen mit kleinem Flächeninhalt
- ...

Clicker Question

$[a, b) = \{x \in \mathbb{R} : a \le x < b\}$

$\{a, a+1, \ldots, b-1\}$
$\parallel$
$[a..b) = \{n \in \mathbb{Z} : a \le n < b\}$

Was ist die Ausgabe für 3SUM mit folgender Eingabe?

$A[0..6) = [1, 0, -1, 3, 2, -1]$

0 1 2 3 4 5

(0,1,2)
(0,1,5)
(2,4,5)

$A[i]$

A `ArrayIndexOutOfBoundsException`

B 0

C 1

D 2

E 3

F 4

G 4.5

H 5

I 6

→ sli.do/cs210

Clicker Question

Was ist die Ausgabe für 3SUM mit folgender Eingabe?

$A[0..6) = [1, 0, -1, 3, 2, -1]$

- **A** ~~`ArrayIndexOutOfBoundsException`~~
- **B** ~~0~~
- **C** ~~1~~
- **D** ~~2~~
- **E** 3 ✓
- **F** ~~4~~
- **G** ~~4.5~~
- **H** ~~5~~
- **I** ~~6~~

→ sli.do/cs210

Einschub: Template für Entwurf von Algorithmen

1. **Algorithmische Idee**
   Kurze abstrakte Idee, die dem Algorithmus zu Grunde liegt (Freitext)
   (ein Experte könnte den Rest von hier ergänzen)

2. **Pseudocode**
   struktierte Beschreibung der Methode inklusive Rand- und Basisfälle
   eindeutig, präzise; nah an echtem Code

3. **Korrektheitsbeweis**
   Argument, warum der Algorithmus das korrekte Ergebnis liefert
   oft per Induktion unter Verwendung von Invarianten

4. **Analyse**
   Analyse der Kosten des Algorithmus
   i. d. R. Θ-Klasse für den Worst Case;
   wo interessant auch Speicherbedarf

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. **Algorithmische Idee**
   Alle Tripel durchprobieren

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. **Algorithmische Idee**
   Alle Tripel durchprobieren

2. **Pseudocode**

```
c = 0
for i = 0, ..., n − 3
    for j = i + 1, ..., n − 2
        for k = j + 1, ..., n − 1
            t = A[i] + A[j] + A[k]
            if t == 0 then c := c + 1
        end for
    end for
end for
return c
```

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. **Algorithmische Idee**
   Alle Tripel durchprobieren

2. **Pseudocode**

```
c = 0
for i = 0, ..., n − 3
    for j = i + 1, ..., n − 2
        for k = j + 1, ..., n − 1
            t = A[i] + A[j] + A[k]
            if t == 0 then c := c + 1
        end for
    end for
end for
return c
```

3. **Korrektheitsbeweis**
   direkt nach Definition von c

$$c = \left|\left\{(i,j,k) : 0 \le i < j < k < n \wedge A[i] + A[j] + A[k] = 0\right\}\right|$$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. **Algorithmische Idee**
   Alle Tripel durchprobieren

2. **Pseudocode**

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. **Korrektheitsbeweis**
   direkt nach Definition von c

Analysen später nochmal im Detail

4. **Analyse**

   # Ausführungen von Zeilen 5–6

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. **Algorithmische Idee**
   Alle Tripel durchprobieren

2. **Pseudocode**

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. **Korrektheitsbeweis**
   direkt nach Definition von c

Analysen später nochmal im Detail

4. **Analyse**

   # Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1$$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
c = 0
for i = 0, ..., n − 3
    for j = i + 1, ..., n − 2
        for k = j + 1, ..., n − 1
            t = A[i] + A[j] + A[k]
            if t == 0 then c := c + 1
        end for
    end for
end for
return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$|[a..b]| = b - a + 1$

$n - (i+1+1)$

$= n - i - 2$

$n - (n-2+1)$

$= 1$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$$= \sum_{i=0}^{n-3} \sum_{j=1}^{n-i-2} j$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2},$$

$1 + 2 + 3 + 4$
$+ 4 + 3 + 2 + 1$
$5 + 5 \cdot \cdot 5$

https://www.wild-inter.net/tcscs.pdf

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$$= \sum_{i=0}^{n-3} \sum_{j=1}^{n-i-2} j = \sum_{i=0}^{n-3} \frac{(n-i-2)(n-i-1)}{2}$$

$n-3$ $\quad$ $n-1$

$$\frac{1 \cdot 2}{2} \qquad \frac{(n-2)(n-1)}{2}$$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$$= \sum_{i=0}^{n-3} \sum_{j=1}^{n-i-2} j = \sum_{i=0}^{n-3} \frac{(n-i-2)(n-i-1)}{2}$$

$$= \frac{1}{2} \sum_{i=1}^{n-2} i(i+1) = \frac{1}{2} \sum_{i=1}^{n-2} i^2 + \frac{1}{2} \sum_{i=1}^{n-2} i$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}, \quad \sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6},$$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$$= \sum_{i=0}^{n-3} \sum_{j=1}^{n-i-2} j = \sum_{i=0}^{n-3} \frac{(n-i-2)(n-i-1)}{2}$$

$$= \frac{1}{2} \sum_{i=1}^{n-2} i(i+1) = \frac{1}{2} \sum_{i=1}^{n-2} i^2 + \frac{1}{2} \sum_{i=1}^{n-2} i$$

$$= \frac{(n-2)(n-1)(2(n-2)+1)}{12} + \frac{(n-2)(n-1)}{4}$$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$$= \sum_{i=0}^{n-3} \sum_{j=1}^{n-i-2} j = \sum_{i=0}^{n-3} \frac{(n-i-2)(n-i-1)}{2}$$

$$= \frac{1}{2} \sum_{i=1}^{n-2} i(i+1) = \frac{1}{2} \sum_{i=1}^{n-2} i^2 + \frac{1}{2} \sum_{i=1}^{n-2} i$$

$$= \frac{(n-2)(n-1)(2(n-2)+1)}{12} + \frac{(n-2)(n-1)}{4}$$

$$= \frac{(n-2)(n-1)n}{6} = \binom{n}{3}$$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$$= \sum_{i=0}^{n-3} \sum_{j=1}^{n-i-2} j = \sum_{i=0}^{n-3} \frac{(n-i-2)(n-i-1)}{2}$$

$$= \frac{1}{2} \sum_{i=1}^{n-2} i(i+1) = \frac{1}{2} \sum_{i=1}^{n-2} i^2 + \frac{1}{2} \sum_{i=1}^{n-2} i$$

$$= \frac{(n-2)(n-1)(2(n-2)+1)}{12} + \frac{(n-2)(n-1)}{4}$$

$$= \frac{(n-2)(n-1)n}{6}$$

Ist das jetzt gut??

Einschub: Überschlagsrechnungen (*back-of-the-envelope math*)

Ob $T(n) = \frac{(n-2)(n-1)n}{6}$ Operationen machbar sind, hängt von n ab!

Ganz grobe Überschlagsrechnungen sind hier hilfreich!

n	$T(n)$	**Zeit bei 1GHz / 1ns pro Operation**
10	120	instant
100	161 700	0.1 ms
1 000	166 167 000	0.16 s
10 000	166 616 670 000	2:40 min
100 000	166 661 666 700 000	46 h (!)

$\rightsquigarrow$ Arrays mit 10 000 Zahlen grade noch machbar, 100 000 eher nicht.

Einschub: Überschlagsrechnungen (*back-of-the-envelope math*)

Ob $T(n) = \dfrac{(n-2)(n-1)n}{6}$ Operationen machbar sind, hängt von n ab!

Ganz grobe Überschlagsrechnungen sind hier hilfreich!

n	$T(n)$	**Zeit bei 1GHz / 1ns pro Operation**
10	120	instant
100	161 700	0.1 ms
1 000	166 167 000	0.16 s
10 000	166 616 670 000	2:40 min
100 000	166 661 666 700 000	46 h (!)

⇝ Arrays mit 10 000 Zahlen grade noch machbar, 100 000 eher nicht.

Übrigens: Man kann 3SUM deutlich schneller lösen . . .
werden wir noch einmal wiedersehen 😉

3.2 Folgen & Grenzwerte

Definitionen zur Erinnerung

Ein bisschen Notation

- $\mathbb{N}_0 = \{0, 1, 2, 3, \ldots, \}$ natürliche Zahlen mit 0
- $\mathbb{N}_{\geq 1} = \{1, 2, 3, \ldots, \}$ natürliche Zahlen ab 1
- $\cancel{\mathbb{N}}$: (Uneindeutigkeit vermeiden)
- $\mathbb{R}$: reelle Zahlen

$\mathbb{Z}$

Definitionen zur Erinnerung

Ein bisschen Notation

- $\mathbb{N}_0 = \{0, 1, 2, 3, \ldots, \}$ natürliche Zahlen mit 0
- $\mathbb{N}_{\geq 1} = \{1, 2, 3, \ldots, \}$ natürliche Zahlen ab 1
- $\cancel{\mathbb{N}}$: (Uneindeutigkeit vermeiden)
- $\mathbb{R}$: reelle Zahlen
- $\lg = \log_2$; $\ln = \log_e$ $e \approx 2.71$ $\cancel{\log_0}$

  ld

Definitionen zur Erinnerung

Ein bisschen Notation

- $\mathbb{N}_0 = \{0, 1, 2, 3, \ldots, \}$ natürliche Zahlen mit 0
- $\mathbb{N}_{\geq 1} = \{1, 2, 3, \ldots, \}$ natürliche Zahlen ab 1
- $\cancel{\mathbb{N}}$: (Uneindeutigkeit vermeiden)
- $\mathbb{R}$: reelle Zahlen
- $\lg = \log_2$; $\ln = \log_e$

Folgen

- (Reelle) *Zahlenfolge* $(a_n)_{n \in \mathbb{N}_{\geq 1}}$
  formal eine Funktion $a : \mathbb{N}_{\geq 1} \to \mathbb{R}$
  schreiben auch "$a_1, a_2, a_3, \ldots$"

Definitionen zur Erinnerung

Ein bisschen Notation

- $\mathbb{N}_0 = \{0, 1, 2, 3, \ldots, \}$ natürliche Zahlen mit 0
- $\mathbb{N}_{\geq 1} = \{1, 2, 3, \ldots, \}$ natürliche Zahlen ab 1
- $\cancel{\mathbb{N}}$: (Uneindeutigkeit vermeiden)
- $\mathbb{R}$: reelle Zahlen
- $\lg = \log_2$; $\ln = \log_e$

Folgen

"$a_n = n^2$" $(a : n \mapsto n^2)$

- (Reelle) *Zahlenfolge* $(a_n)_{n \in \mathbb{N}_{\geq 1}}$
  formal eine Funktion $a : \mathbb{N}_{\geq 1} \to \mathbb{R}$
  schreiben auch "$a_1, a_2, a_3, \ldots$"
- *Grenzwert* einer Folge
  $\lim\limits_{n \to \infty} a_n = a^*$ genau dann, wenn $\boxed{\overset{\text{für alle}}{\forall \varepsilon > 0} \ \overset{\text{existiert}}{\exists n_0 \in \mathbb{N}_{\geq 1}} \ \forall n \geq n_0 : |a_n - a^*| < \varepsilon}$

Definitionen zur Erinnerung

Ein bisschen Notation

- $\mathbb{N}_0 = \{0, 1, 2, 3, \ldots, \}$ natürliche Zahlen mit 0
- $\mathbb{N}_{\geq 1} = \{1, 2, 3, \ldots, \}$ natürliche Zahlen ab 1
- $\cancel{\mathbb{N}}$: (Uneindeutigkeit vermeiden)
- $\mathbb{R}$: reelle Zahlen
- $\lg = \log_2$; $\ln = \log_e$

Folgen

$a_n = (-1)^n$

- (Reelle) *Zahlenfolge* $(a_n)_{n \in \mathbb{N}_{\geq 1}}$
  formal eine Funktion $a : \mathbb{N}_{\geq 1} \to \mathbb{R}$
  schreiben auch "$a_1, a_2, a_3, \ldots$"
- *Grenzwert* einer Folge
  $\lim\limits_{n \to \infty} a_n = a^*$ genau dann, wenn $\boxed{\forall \varepsilon > 0\ \exists n_0 \in \mathbb{N}_{\geq 1}\ \forall n \geq n_0 : |a_n - a^*| < \varepsilon}$
- Nicht jede Folge hat einen Grenzwert, aber wenn er existiert, dann ist er eindeutig.

Folgen – Beispiel

$$\boxed{\forall \varepsilon > 0\ \exists n_0 \in \mathbb{N}_{\geq 1}\ \forall n \geq n_0 : |a_n - a^*| < \varepsilon}$$

Es sei die Folge $(a_n)_{n \in \mathbb{N}_{\geq 1}}$ mit $a_n = \dfrac{1}{n}$ gegeben.

Es gibt: $\lim\limits_{n \to \infty} a_n = 0$

$\varepsilon > 0$ beliebig. Wählen $n_0 := \left\lceil \frac{1}{\varepsilon} \right\rceil + 1$

$\Rightarrow$ für $n \geq n_0$ beliebig gilt:

$$|a_n - 0| = \frac{1}{n} \underset{n \geq n_0}{\leq} \frac{1}{n_0} = \frac{1}{\left\lceil \frac{1}{\varepsilon} \right\rceil + 1}$$

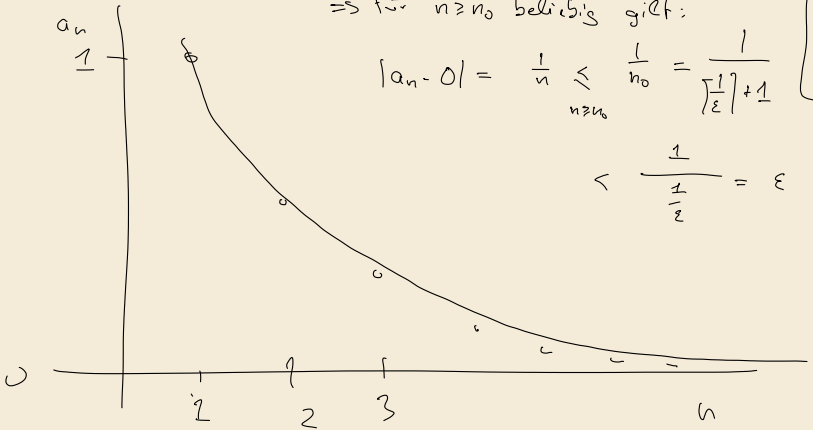
$$< \frac{1}{\frac{1}{\varepsilon}} = \varepsilon \qquad \square.$$

N.R.

$|a_n - 0| < \varepsilon$

$= \frac{1}{n} \overset{!}{<} \varepsilon$

$n > \frac{1}{\varepsilon}$ tut's



Grenzwerte von Funktionen

$f : \mathbb{R} \to \mathbb{R}$

"ε-δ-Kriterium"

$$\lim_{x \to \xi} f(x) = f^* \text{ gdw } \forall \varepsilon > 0 \; \exists \delta > 0 \; \forall x \in \mathbb{R} : |x - \xi| < \delta \implies |f(x) - f^*| < \varepsilon$$

Limes superior, Limes inferior

- Supremum ist die kleinste obere Schranke
  Für $M \subseteq \mathbb{R}$ ist $\sup M = \min\{b \in \mathbb{R} : \forall x \in M : x \leq b\}$

$$M = \left\{ 1 - \frac{1}{n} : n \in \mathbb{N}_0 \right\}$$

$$x \in M \Rightarrow x \leq 1$$

$$\sup M = 1$$

Limes superior, Limes inferior

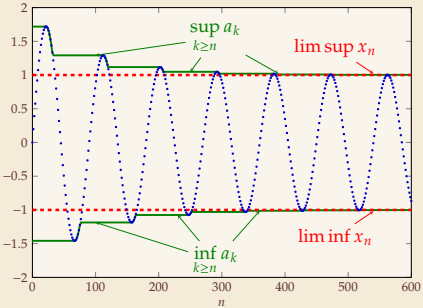
- Supremum ist die kleinste obere Schranke
  Für $M \subseteq \mathbb{R}$ ist $\sup M = \min\{b \in \mathbb{R} \,:\, \forall x \in M : x \leq b\}$

$\inf\left\{\frac{1}{n} \,;\, n \in \mathbb{N}\right\} = 0$

- Analog ist Infimum die größte untere Schranke
- Beide müssen nicht existieren ($+\infty$ bzw. $-\infty$); wenn sie existieren, sind sie eindeutig.

Limes superior, Limes inferior

- Supremum ist die kleinste obere Schranke
  Für $M \subseteq \mathbb{R}$ ist $\sup M = \min\{b \in \mathbb{R} \,:\, \forall x \in M : x \leq b\}$
- Analog ist Infimum die größte untere Schranke
- Beide müssen nicht existieren ($+\infty$ bzw. $-\infty$); wenn sie existieren, sind sie eindeutig.



- Für Folge $(a_n)_{n \in \mathbb{N}_{\geq 1}}$ ist der *Limes superior*:

$$\boxed{\limsup_{n \to \infty} a_n \;=\; \inf\Big\{\sup\big\{a_k : k \geq n\big\} : n \in \mathbb{N}_{\geq 1}\Big\}}$$

Äquivalent: größter Häufungspunkt von (a_n);
beste obere Schranke für a_n
für genügend große n

- Analog Limes inferior: $\liminf\limits_{n \to \infty} a_n \;=\; \sup\Big\{\inf\big\{a_k : k \geq n\big\} : n \in \mathbb{N}_{\geq 1}\Big\}$

3.3 Asymptotische Notation

Clicker Question

Ist $n^{0.1} = O\big((\log(n))^{10}\big)$?

- **A** O?? nie gesehen
- **B** Ja (bin mir sicher).
- **C** Nein (bin mir sicher).
- **D** Denke Ja
- **E** Denke Nein
- **F** keine Ahnung

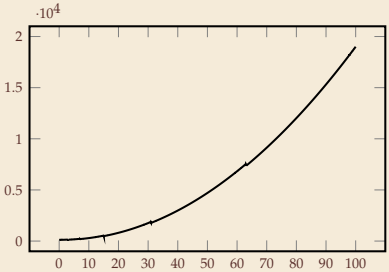
→ sli.do/cs210

Asymptotische Vergleiche

Asymptotik = Approximation um ∞

Beispiel: Betrachte Funktion $f(n)$ mit

$2n^2 - 3n\lfloor\log_2(n+1)\rfloor + 7n - 3\lfloor\log_2(n+1)\rfloor + 120$

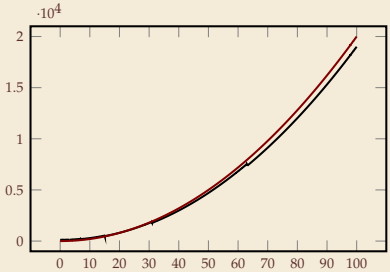


Asymptotische Vergleiche

Asymptotik = Approximation um ∞

Beispiel: Betrachte Funktion $f(n)$ mit

$$2n^2 - 3n\lfloor\log_2(n+1)\rfloor + 7n - 3\lfloor\log_2(n+1)\rfloor + 120 \sim 2n^2$$



Asymptotische Approximationen in der Analyse von Algorithmen

- abstrahiert von unwichtigen Details
- vereinfacht die Analyse selbst
- ermöglicht oft erst konklusiven Vergleich

Asymptotische Notation – Formale Definition

- **"Tilde Notation":** $f(n) \sim g(n)$ gdw $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 1$

  genau dann, wenn (*iff*)

  „f und g sind *asymptotisch äquivalent*"

Asymptotische Notation – Formale Definition

- **"Tilde Notation":** $f(n) \sim g(n)$ gdw $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 1$

  genau dann, wenn (*iff*)

  „f und g sind *asymptotisch äquivalent*"

- **"Big-Oh Notation":** $f(n) \in O\big(g(n)\big)$ gdw $\left|\frac{f(n)}{g(n)}\right|$ beschränkt für $n \geq n_0$

  schreiben auch '=' stattdessen

  brauchen Supremum da Limes evtl. nicht existiert!

  gdw $\lim_{n\to\infty} \sup \left|\frac{f(n)}{g(n)}\right| < \infty$

Varianten:

- $f(n) \in \Omega\big(g(n)\big)$ gdw $g(n) \in O\big(f(n)\big)$

  "Big-Omega"

- $f(n) \in \Theta\big(g(n)\big)$ gdw $f(n) \in O\big(g(n)\big)$ **und** $f(n) \in \Omega\big(g(n)\big)$

  "Big-Theta"

Asymptotische Notation – Formale Definition

- **"Tilde Notation":** $f(n) \sim g(n)$ gdw $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 1$.

  genau dann, wenn (*iff*)

  „f und g sind *asymptotisch äquivalent*"

- **"Big-Oh Notation":** $f(n) \in O\big(g(n)\big)$ gdw $\left|\frac{f(n)}{g(n)}\right|$ beschränkt für $n \geq n_0$

  schreiben auch '=' stattdessen

  gdw $\limsup_{n\to\infty} \left|\frac{f(n)}{g(n)}\right| < \infty$

  brauchen Supremum da Limes evtl. nicht existiert!

  Varianten:

  - $f(n) \in \Omega\big(g(n)\big)$ gdw $g(n) \in O\big(f(n)\big)$

    "Big-Omega"

  - $f(n) \in \Theta\big(g(n)\big)$ gdw $f(n) \in O\big(g(n)\big)$ **und** $f(n) \in \Omega\big(g(n)\big)$

    "Big-Theta"

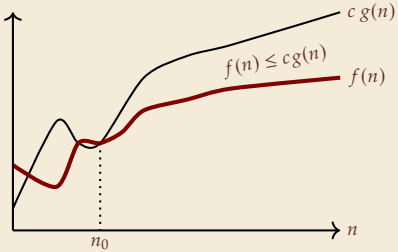
- **"Little-Oh Notation":** $f(n) \in o\big(g(n)\big)$ gdw $\lim_{n\to\infty} \left|\frac{f(n)}{g(n)}\right| = 0$

  analog: $f(n) \in \omega\big(g(n)\big)$ falls $|f(n)/g(n)| \to \infty$

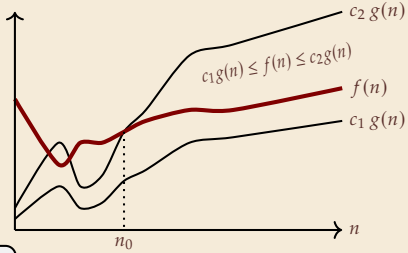
(Vorteil dieser Definition: Funktioniert genauso für $f, g : \mathbb{R} \to \mathbb{R}$ und für andere Grenzprozesse als $n \to \infty$)

Asymptotische Notation – Intuition

- $f(n) = O(g(n))$: $f(n)$ ist **höchstens** $g(n)$
  bis auf konstante Faktoren
  und
  für hinreichend große n



- $f(n) = \Theta(g(n))$: $f(n)$ ist **gleich** $g(n)$
  bis auf konstante Faktoren
  und
  für hinreichend große n



Plots können irreführend sein!

Beispiel

O-Notation – Beispiel

$f(n) = 3n^3 + 2n^2, \quad g(n) = n^3$

$$\frac{f(n)}{g(n)} = \frac{3n^3 + 2n^2}{n^3} = 3 + \underbrace{\frac{2}{n}}_{\to 0}$$

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 3$$

$f \not\sim g$

$f(n) = O(g(n))$ da 3 beschränkt

$g(n) \overset{?}{=} O(f(n))$ ✓ $\quad \frac{g(n)}{f(n)} \to \frac{1}{3}$

$\Rightarrow f(n) = \Omega(g(n))$

$\Rightarrow f(n) = \Theta(g(n))$